

LTL-STL Assignment

Assignment 1 - MoSiS

Mehmet Tuna Ozkalkanli - s0261404
Roman Zalutskyi - s0260759

University of Antwerp
October 2026

Contents

1 Introduction **3**

2 Implementation **3**

2.1 LTL Systems 4

2.1.1 Simple traffic light 4

2.1.2 Simple production system 5

2.1.3 Pipelined production system 8

2.2 STL Systems 10

2.2.1 Cruise control 10

2.2.2 Gantry crane 11

2.2.3 Water heating 13

1 Introduction

This report presents the first assignment of the MoSiS course, titled "LTL and STL assignment". The main goal is to translate the given requirements into temporal logic formulas and then check whether the provided traces satisfy these formulas.

The analyzed systems include a simple traffic light, a simple production system, a pipelined production system, a cruise control system, a gantry crane system and a water heating system.

These verifications were handled with Python, using the `metric-temporal-logic` and `pandas` libraries. Git and GitHub were used for version control and for sharing and reviewing the implementation during development.

For each system, relevant data from the traces were extracted, the properties were defined and the satisfaction was evaluated for each trace.

The remainder of this report describes the implementation approach, the formalization of the properties and the verification results.

2 Implementation

The verification process was implemented in Python. For each system, the provided CSV files were read with the help of the `pandas` library and then were converted into traces for the `metric-temporal-logic` library for the implementation of the formulas.

For the LTL systems, the column `step` was used as the time index and the boolean values in the CSV files were used as the atomic propositions.

For the STL systems, since some properties depend on comparisons, boolean signals were created before evaluating the formulas, as shown in the assignment instructions.

For example, A real trace from the assignment checking if the intervehicular distance is between 0.5 meters of 10 meters :

```
"necessary_gap" :
list(zip(frame["time"], (10 - frame["intervehicular_distance.y"]).abs() <= 0.5))
```

After defining the traces and the TL formulas, each property was evaluated for every trace. The results were then exported into CSV files

2.1 LTL Systems

2.1.1 Simple traffic light

The following LTL properties were formalized and evaluated against the traffic light simulation traces. To respect the strict parsing rules of the `pyMTL` library, complex operations are nested with explicit inner parentheses.

one_lamp : Ensures that at every single step, exactly one of the three lights (red, yellow, or green) is active.

```
G[0,201] (
    ((red & ~yellow) & ~green)
    | ((~red & yellow) & ~green)
    | ((~red & ~yellow) & green)
)
```

The three exclusive states are evaluated individually and combined with logical OR. The `G` operator enforces this condition across all steps.

starts_red : Checks if the traffic light initially starts in red at step 0.

```
G[0, 1] red
```

By evaluating the global operator over the interval `[0, 1[`, we check only the initial state without constraining the rest of the trace.

always_red : Verifies if the traffic light stays stuck on red for the entire sequence.

```
G[0,201] red
```

eventually_green : Ensures that the green light turns on at least once at some point down the line.

```
F[0,201] green
```

red_stays_or_next_yellow : Ensures that if a step is red, the immediate next step must either remain red or change over to yellow.

```
G[0,200] (red -> (X red | X yellow))
```

The time boundary is limited to `[0,200]` to ensure that the next operator (`X`) always lands within a valid recorded index of the trace.

no_red_to_green : Guarantees that a red step is never directly followed by a green step.

```
G[0,200] (red -> ~X green)
```

no_green_to_red : Guarantees that a green step is never followed immediately by a red step.

```
G[0,200] (green -> ~X red)
```

yellow_opens_to_green : If the light is yellow and the sequence direction is moving forward (`alternate` is false), then when the light stops being yellow, the next step must be green.

```
G[0,200] (((yellow & ~alternate) & ~X yellow) -> X green)
```

yellow_returns_to_red : If the light is yellow and the sequence direction is moving backward (`alternate` is true), then when the light stops being yellow, the next step must return to red.

```
G[0,200] (((yellow & alternate) & ~X yellow) -> X red)
```

yellow_after_red_may_wait : Whenever the traffic light switches from red to yellow, the yellow light should hold until green appears, or keep holding indefinitely if green never shows up.

```
G[0, 200] ((red & X yellow) -> X (yellow W green))
```

An implication maps the transition from red to yellow, and the weak-until operator (`W`) elegantly covers the case where the trace cuts off before green ever triggers.

2.1.2 Simple production system

The following LTL properties were used to verify the provided traces:

m1_one_mode : Ensures m1 is always exactly in one of its four modes.

```
"G[0,201] ("
        "(m1_busy & ~m1_down & ~m1_idle & ~m1_repair) "
        "| (~m1_busy & m1_down & ~m1_idle & ~m1_repair)"
        "| (~m1_busy & ~m1_down & m1_idle & ~m1_repair) "
        "| (~m1_busy & ~m1_down & ~m1_idle & m1_repair)"
        ")"
```

The four possible cases were combined with OR while the other three should be false in every case globally from step 0 to 200

m2_one_mode : Follows the same logic as **m1_one_mode**

max_one_part : Ensures either the workshop is empty or it's processing a part.

```
"G[0,201] ("
        "(idle & ~p_in & ~p_m1 & ~p_m2 & ~p_out) | "
        "(~idle & p_in & ~p_m1 & ~p_m2 & ~p_out) | "
        "(~idle & ~p_in & p_m1 & ~p_m2 & ~p_out) | "
        "(~idle & ~p_in & ~p_m1 & p_m2 & ~p_out) | "
        "(~idle & ~p_in & ~p_m1 & ~p_m2 & p_out)"
        ")"
```

Follows a very similar logic to `m_one_mode`: either the system is idle or the part is located exclusively at one stage of the production process.

not_both_in_repair : Ensures that m1 and m2 are never getting repaired simultaneously

```
"G[0,201] (~(m1_repair & m2_repair))"
```

The conjunction checks if m1 and m2 are in repair at the same time. It is negated, and the `G` operator makes sure that this condition holds for every step.

part_at_m1_mode : Ensures if m1 has the part, it should either be in one of its 3 active modes and not idle

```
"G[0,201] (p_m1 -> ((m1_busy | m1_down | m1_repair) & ~(m1_idle)))"
```

The implication checks if m1 has the part. If it has the part, m1 should be either busy, down or in repair and not idle `G` operator makes sure that this condition is checked at every step of the trace.

machines_not_both_holding : Ensures that m1 and m2 do not hold the part at the same time.

```
G[0,201] (p_m1 -> ~(p_m2))
```

The implication checks whether the part is on m1. If p_m1 is true then p_m2 must be false. The `G` operator makes sure that this condition is checked at every step.

m1_before_m2 : Ensures that a part cannot reach machine 2 before it has reached machine 1.

```
G[0,201] (p_in -> (~p_m2 W p_m1))
```

Whenever a new part enters the system, p_m2 must remain false until p_m1 becomes true. TThe weak-until operator `W` is used because the property can still hold if `p_m1` never becomes true, as long as `p_m2` also remains false. The `G` operator applies this check throughout the whole trace.

eventually_shipped : Ensures that the part eventually reaches the output of the system.

```
F[0,201] p_out
```

The `F` operator checks that p_out eFentually becomes true between step 0 and step 200.

wait_may_still_continue : Ensures that the workshop is idle until a new part arrives or the end of the trace (if a new part doesn't arrive)

```
"G[0,200] (p_out -> X (idle W p_in))"
```

Whenever the current state is p_out the next state should be idle until p_in holds. p_in may never arrive and the system could remain idle until the end of the system so we use the weak until operator.

2.1.3 Pipelined production system

For the pipelined production system, the properties focus on tracking stage changes and concurrent operations since multiple parts can exist inside the workshop simultaneously. The formulas were formalized and checked using the following LTL logic:

m1_one_mode : Ensures that machine 1 is always in exactly one of its four designated behavioral modes at any point in the trace.

```
G[0,201] (
    (m1_busy & ~m1_down & ~m1_idle & ~m1_repair)
    | (~m1_busy & m1_down & ~m1_idle & ~m1_repair)
    | (~m1_busy & ~m1_down & m1_idle & ~m1_repair)
    | (~m1_busy & ~m1_down & ~m1_idle & m1_repair)
)
```

This matches the mode logic used in the simple production system, ensuring exclusive states across all 201 steps.

not_both_in_repair : Guarantees that machine 1 and machine 2 are never under repair at the exact same step.

```
G[0,201] (~(m1_repair & m2_repair))
```

both_machines_holding : Verifies whether there is at least one instance in the trace where both machine 1 and machine 2 hold a part simultaneously.

```
F[0,201] (p_m1 & p_m2)
```

The `F` operator is used here to identify if this pipelined concurrence scenario ever triggers.

m1_while_buffer_full : Checks if machine 1 holds a part at the same time that the intermediate storage buffer is full.

```
F[0,201] (p_m1 & p_buffer)
```

m1_to_buffer : Ensures that a part only exits machine 1 when the machine is busy and the downstream buffer is clear, landing a part in the buffer on the next step.

```
G[0,200] ((p_m1 & ~X p_m1) -> ((m1_busy & ~p_buffer) & X p_buffer))
```

This formula explicitly assumes that a part leaving machine 1 always leaves that specific stage empty for at least one clock cycle.

ship_removes_only_out : Ensures that when a finished product is shipped out of the system, all other storage states (`p_in`, `p_m1`, `p_buffer`, and `p_m2`) are entirely unaffected and retain their state values.

```
G[0,200] (
    (p_out & ~X p_out) -> (
        (p_in <-> X p_in) & (p_m1 <-> X p_m1)
        & (p_buffer <-> X p_buffer) & (p_m2 <-> X p_m2)
    )
)
```

A biconditional structural equivalence operator (`<->`) is applied here to map state preservation across sequential boundaries.

arrival_only_fills_in : Ensures that when a brand new raw part arrives at the input, all internal downstream processing stages are left unaltered.

```
G[0,200] (
```

```
    (~p_in & X p_in) -> (
        (p_m1 <-> X p_m1) & (p_buffer <-> X p_buffer)
        & (p_m2 <-> X p_m2) & (p_out <-> X p_out)
    )
)
```

2.2 STL Systems

2.2.1 Cruise control

For the cruise control system, some values in the traces were first converted into boolean propositions according to the requirements. The following propositions were used:

- `safe_space`: the intervehicular distance is at least 9 meters, $d \geq 9$
- `safe_accel`: the absolute acceleration of the ego car is at most 10, $|a| \leq 10$
- `necessary_gap`: the intervehicular distance is within 0.5 meters of 10 meters, $|10 - d| \leq 0.5$
- `settled_loose`: the intervehicular distance is within 1 meter of 10 meters, $|10 - d| \leq 1$

reaches_soon : Ensures that the intervehicular distance is within 0.5 m of 10 within the first 20 seconds

```
F[0,20.1] necessary_gap
```

The `F` operator is used because the condition only needs to become true at least once during the first 20 seconds.

settled_tight : Ensures that from 30 seconds until the end of the trace, the intervehicular distance remains within 0.5 m of 10.

```
G[30,60.1] necessary_gap
```

The `G` operator is used because the condition must remain true from the 30 second mark until the end.

settled_loose : Ensures that from 20 seconds until the end of the trace, the intervehicular distance remains within 1 meter of 10.

```
G[20,60.1] settled_loose
```

keeps_distance Ensures that the distance between the two cars never goes below 9 meters.

```
G[0,60.1] safe_space
```

The globally operator makes sure that the minimum distance requirement holds during the complete trace.

accel_bounded Ensures that the absolute acceleration of the ego car never exceeds 10.

```
G[0,60.1] safe_accel
```

Since this is a safety requirement, the `G` operator is used to check it for the entire trace.

2.2.2 Gantry crane

For the gantry crane system, continuous signals in the traces were first processed into boolean propositions using thresholds defined by the physical limits of the system. Each simulation trace spans a total of 20 seconds. The following propositions were built:

- `angle_half`: the absolute sway angle of the pendulum is at most 0.5 radians, $|\theta| \leq 0.5$
- `angle_limit`: the absolute sway angle of the pendulum is at most 1.5 radians, $|\theta| \leq 1.5$
- `reached_target`: the trolley position reaches or exceeds 9.5 meters, $x \geq 9.5$
- `is_settled`: the trolley position stays stabilized within a 0.5-meter window around the target position, $9.5 \leq x \leq 10.5$
- `under_overshoot`: the trolley position stays within the tracking boundary and does not exceed 10.5 meters, $x \leq 10.5$

angle_within_half : Ensures that the absolute pendulum sway angle stays within 0.5 radians over the entire simulation trace.

```
G[0, 20.1] angle_half
```

The globally operator evaluates this constraint across the whole timeframe to verify that the crane movement keeps the pendulum stable.

angle_within_limit : Ensures that the absolute sway angle never surpasses the maximum safety boundary of 1.5 radians.

```
G[0, 20.1] angle_limit
```

reaches_by_8s : Ensures that the trolley position successfully crosses into the target zone (at least 9.5 meters) within the first 8 seconds.

```
F[0, 8.01] reached_target
```

An eventually operator (`F`) is used here since the condition only needs to be satisfied at least once during the specified 8-second interval.

settled : Ensures that the trolley position remains converged and completely settled within the target window from the 10-second mark until the end of the simulation.

```
G[10, 20.1] is_settled
```

The globally operator enforces that the trolley remains steadily within bounds during the final half of the runtime.

no_overshoot : Ensures that the trolley position never extends past 10.5 meters due to physical rail length constraints.

```
G[0, 20.1] under_overshoot
```

2.2.3 Water heating

For the water heating system, boolean propositions were created from temperature, tank level, and heater power signals.

- `reaches_setpoint`: the outlet temperature is at least 321.15 K, $T \geq 321.15$
- `safe_temp`: the outlet temperature is at most 330 K, $T \leq 330$
- `settled`: the outlet temperature is within 3 K of 323.15 K, $|323.15 - T| \leq 3$
- `level_low`: the tank level is at least 1.4, level ≥ 1.4
- `level_high`: the tank level is at most 2.9, level ≤ 2.9
- `heater_on`: the heater power is at least 150, heater_power ≥ 150

reaches_setpoint : Ensures that the outlet temperature eventually reaches at least 321.15 K.

```
F[0,86400.1] reaches_setpoint
```

The `F` operator is used because the required temperature only needs to be reached at least once during the trace.

no_temperature_overshoot : Ensures that the outlet temperature never exceeds 330 K.

```
G[0,86400.1] safe_temp
```

The `G` operator checks that the condition holds throughout the complete simulation.

settled_after_half_day : Ensures from 43200 on the outlet temperature remains within 3 K of 323.15 K.

```
G[43200,86400.1] settled
```

The condition is checked globally from 43200 seconds until the end of the one-day simulation.

level_in_band : Ensures that the tank level always remains between 1.4 and 2.9.

```
G[0,86400.1] (level_low & level_high)
```

Both the lower and upper level conditions must be true at the same time. This also alternatively could've been done with a boolean proposition that checks both bounds directly in the trace rather than using two boolean propositions

heater_stays_on : Ensures that the heater power remains at least 150 during the complete simulation.

```
G[0,86400.1] heater_on
```

AI Usage Statement

Mehmet Tuna Özkalkanlı, Roman Zalutskyi: Gen AI was used for LaTeX syntax assistance and language corrections. It was not used to generate the technical solutions, temporal logic formulas or implementation.

References

[1] University of Antwerp, *Modelling of Software-Intensive Systems (MoSiS)*. `http://msdl.uantwerpen.be/people/hv/teaching/MoSIS/lectures/presentation.LTL_STL.pdf` `http://msdl.uantwerpen.be/people/hv/teaching/MoSIS/assignments/LTL-STL`

[2] `metric-temporal-logic` library documentation. `https://github.com/mvcisback/py-metric-temporal-logic`

[3] `pandas` library documentation. `https://pandas.pydata.org/docs/index.html`